

Connect PHP to MySQL with PDO

Goal: Reach CRUD

CRUD

↑

PHP application logic

↕ PDO

MySQL persistent data

Today, we build that PHP-to-MySQL connection so our application can perform CRUD operations.

IMPORTANT

Who Does What?

CRUD is the four basic operations used to manage stored data: create, read, update, and delete.

- The **PHP server** receives the user's request and runs the application logic.
- **PDO** carries database operations between PHP and MySQL.
- **MySQL** stores, finds, changes, and removes the data.

Create and Select the Database

In the MySQL shell (`mysql>`):

```
CREATE DATABASE IF NOT EXISTS studentdb;  
USE studentdb;
```

- A **database** groups related tables.
- `USE studentdb` selects it for the commands that follow.

```
SHOW DATABASES;
```

Confirm that `studentdb` appears in the result.

The **students** Schema - Database Design

Database design means deciding what information to store and how to organize it before the application saves data.

- A **schema** is the blueprint for a table: its columns, data types, and rules.

The **students** Table Design

Column	Stores	Rule
id	Student identifier	Unique; generated automatically
name , email	Contact information	Text
age , year	Numeric information	Number
major	Field of study	Text

Create the **students** Table

Implement the schema with this SQL command:

```
CREATE TABLE IF NOT EXISTS students (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100),  
    email VARCHAR(150),  
    age INT,  
    major VARCHAR(100),  
    year INT  
);
```

Check the Table

```
SHOW TABLES;  
DESCRIBE students;
```

This same student data model will continue into the REST API lesson.

```
id, name, email, age, major, year
```

The database is now ready to support CRUD.

IMPORTANT

Create a User for the PHP Application

Run these commands in the MySQL shell:

```
CREATE USER IF NOT EXISTS 'ase230'@'localhost'  
  IDENTIFIED BY 'ase230pass';  
ALTER USER 'ase230'@'localhost'  
  IDENTIFIED BY 'ase230pass';  
GRANT SELECT, INSERT, UPDATE, DELETE ON studentdb.*  
  TO 'ase230'@'localhost';
```

- Use a separate MySQL account for PHP; do not connect the application as `root`.
- `IF NOT EXISTS` and `ALTER USER` let you re-run this safely if you make a mistake.

What These User Commands Do

Command	Meaning
<code>CREATE USER</code>	Creates the MySQL account <code>ase230</code> that PHP will use.
<code>IDENTIFIED BY</code> <code>'ase230pass'</code>	Sets the MySQL password for <code>ase230</code> . It is not your macOS, Linux, or browser password.
<code>'ase230'@'localhost'</code>	Allows this account to connect only from the same computer.
<code>GRANT ... ON</code> <code>studentdb.*</code>	Allows the account to perform CRUD operations on every table in <code>studentdb</code> .

Later, PHP sends `ase230` and `ase230pass` to MySQL through PDO. MySQL accepts the connection only when both values match this account.

PDO Connects PHP to MySQL

PDO means *PHP Data Objects*.

PHP code → PDO → MySQL server

In this course, use PDO for every database operation:

```
$pdo = new PDO($dsn, $username, $password);
```

We will use the same `$pdo` connection for Create, Read, Update, and Delete.

Make the PDO Connection

This is the connection section at the beginning of `index_pdo.php`.

```
<?php
$host = "localhost";
$dbname = "studentdb";
$username = "ase230";
$password = "ase230pass";

$dsn = "mysql:host=$host;dbname=$dbname;charset=utf8mb4";
$pdo = new PDO($dsn, $username, $password, [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
]);

echo "Connected successfully<br>";
```

Run the PHP Server

In your terminal (from the `ase230` course directory):

```
cd module1/code/2_Connect_PHP_with_MySQL  
php -S localhost:8000
```

Then open in your browser:

```
http://localhost:8000/index_pdo.php
```

Connection Checkpoint

`index_pdo.php` connects and then inserts one student, so the browser shows:

```
Connected successfully  
Created Student 1 with ID <number>
```

The ID number may be different — it depends on how many students already exist in the table.

If you see this, the complete path works:

Browser → PHP server → PDO → MySQL server

Driver and connection troubleshooting is in document 4 (Optional PDO Troubleshooting and MySQLi).

First CRUD Operation: Create

This is the second half of `index_pdo.php` — the same file as the connection above.

```
$sql = "INSERT INTO students (name, email, age, major, year)
      VALUES (:name, :email, :age, :major, :year)";

$stmt = $pdo->prepare($sql);
$stmt->execute([
    ':name' => 'Student 1',
    ':email' => 'student1@example.com',
    ':age' => 20,
    ':major' => 'Computer Science',
    ':year' => 2,
]);
```

IMPORTANT

The PDO Pattern Starts Here

```
$sql = "...";  
$stmt = $pdo->prepare($sql);  
$stmt->execute([ /* values */ ]);
```

You will repeat this pattern for CRUD:

- **INSERT** → Create
- **SELECT** → Read
- **UPDATE** → Update
- **DELETE** → Delete

Next, we will implement all four operations.